

GTI525 - Laboratoire #02

Code dorsale et API



**ÉCOLE DE
TECHNOLOGIE
SUPÉRIEURE**

Université du Québec

Étudiants	Jean-Félix ST-HILAIRE
	Gaël FORTIER
	Jamal ATTOU
	Steven LEBLANC-LAFOND
Code Permanents	STHJ09099500
	FORG82860104
	ATTJ63300201
	LEBS22129401
Cours	GTI525
Groupe	01
Session	Été 2024
Charge de laboratoire	Amna Snene
Date	11 juillet 2024

Introduction

Au cours du second laboratoire nous devons intégrer l'architecture de la partie dorsale (*backend*) de notre application et ajouter de nouvelles fonctionnalités à cette dernière. Parmi ces nouvelles fonctionnalités on retrouve la possibilité d'obtenir des prévisions météorologiques pour une station donnée ainsi que les détails d'une météo pour une date précise donnée passée. Nous aborderons dans ce rapport les tenants et aboutissants de ce second laboratoire à travers les différentes tâches effectuées, les difficultés rencontrées et les solutions apportées au projet.

Organisation de l'équipe

Nous avons séparé les tâches du projet selon la séparation définie dans le document de livrable du laboratoire. En essayant un partage équitable des tâches nous avons la séparation suivante :

Tâches	Définition	Personne(s)
T1	Intégration de l'application dorsale	Jamal
T2	Informations météo pour une journée donnée passée	Gael et Jean-Félix
T3	Prévisions météo (pour aujourd'hui et les prochains jours)	Steven et Jean-Félix
T4	Rapport	Tous

1. Architecture logicielle

Afin de conserver une structure facile à manipuler nous avons décidé de diviser le projet en deux dossiers distincts. Cela nous permet entre autres de gérer les packages et librairies spécifiques à la partie frontale ou dorsale. Il y a donc un package module dans chacun des dossiers. Le premier dossier *client* est destiné exclusivement à l'application frontale (*frontend*) et ses librairies. C'est dans ce dossier que nous retrouvons l'intégralité du laboratoire 1 avec de nouveaux ajouts. Le second dossier *server* va contenir l'ensemble du code et des librairies dédiées à l'application dorsale ajoutée au cours du laboratoire.

Front-End (Lab01)

Nous reprenons le code issu du premier laboratoire avec son cadriciel [Vue.JS](#) et la librairie [Bootstrap](#). Les vrais changements apportés concernent les deux nouveaux *components* apportés à savoir *ForecastWeek.vue* et *PastHourlyForecast.vue*

Informations météo pour une journée donnée passée

PastHourlyForecast.html

Ce fichier HTML sert à disposer les éléments et contrôles visuels sur la page web. Dans la page, nous avons un sélecteur pour l'année, le mois ainsi que la date. À chaque changement, l'onglet se rafraîchit et indique à l'utilisateur que la page est en train de charger les données depuis le serveur jusqu'à ce que la page client reçoive les données. Dû à notre conception initiale de notre interface, nous avons rencontrés des bugs avec Vue.js où certaines variables intégrées au HTML ne se mettaient pas à jour. Ainsi, nous avons dû injecter directement le HTML de la table des prévisions en manipulant le DOM (autrement dit, sans passer par `v-html = "..."` de Vue.js).

PastHourlyForecast.vue

Fichier requis par Vue.js pour associer un `.js` et un `.css` à un `.html`. Cela nous permet d'utiliser une balise HTML dans d'autres fichiers `.html` qui représente notre composante Vue.

Par exemple: `<PastHourlyForecast/>` qui se retrouve dans `DataStatMenu.html`, vient intégrer notre composant `PastHourlyForecast` dans le composant parent `DataStatMenu`.

PastHourlyForecast.js

Méthode(s):

- `search()`

Cette méthode lit la date sélectionnée et envoie une requête au serveur comportant l'identifiant de la station sélectionnée ainsi que la date. Le serveur doit normalement envoyer une réponse contenant l'entête des colonnes du tableau à afficher ainsi que les données à afficher. Si aucune rangée n'a été reçue, l'interface affiche un message suggérant que l'utilisateur choisisse une autre date.

Prevision:

ForecastWeek.html

Ce fichier html est utilisé comme « template » pour le fichier de vue. Il consiste d'un « div » avec un « ID = Forecastweek-result » et une commande « `v-html=ForecastHtmlWeek` » afin d'intégrer le résultat remis par le fichier `ForeWeek.js`

ForecastWeek.vue

Cette vue consiste de rassembler le template html qui se retrouve dans « `ForecastWeek.html` », le code Javascript associé (nommé `ForecastWeek.js`) et les fichier CSS de `Vue.css` et `main.css`.

ForecastWeek.js

Ce fichier s'occupe de l'intégration des prévisions sur la page « Prévision ». Pour expliquer en détails, on importe une classe créer spécifiquement pour intégrer les informations obtenues du « Back-end » dans une tab de façon organisé. Cette classe « `ParagraphUtil` » s'occupe aussi des générations d'erreurs. On commence la classe par initialiser la valeur du `ForecastHtmlWeek` (qui est relié directement au `ForecastHtmlWeek` dans le fichier html) avec une erreur parce qu'on assume que s'il n'y a pas d'appel de la fonction « `getForecastView` », il doit avoir une erreur de disponibilité.

Méthode(s) :

- `getForecastView(stationID)`

Dans cette méthode, on demande d'avoir l'identification de la station afin de faire la bonne demande dans le « Back-end ». De plus, on assigne `getForecastView` comme fonction « `async` ». On fait la demande en utilisant un « `await fetch` » (la raison pourquoi on utilise `async`) pour faire la demande dans le « Back-end », on inclut l'identification de la station dans la demande. Quand on reçoit le résultat, on vérifie que la valeur est bien retourné. Si oui, on intègre l'information dans une valeur « `data` »

» et on l'utilise pour organiser une valeur tableau. On utilise le tableau avec « ParagraphUtil » et « getForecastViewHeader » pour créer un texte qui serait retourné à la vue.

- `getForecastViewHeader()`

Renvoie l'en-tête de chaque ligne.

Back-End

Informations météo pour une journée donnée passée

StationPastHourlyForecast.js

Ce fichier contient une classe s'occupant de faire une requête à l'adresse

https://climate.weather.gc.ca/climate_data/bulk_data_e.html (avec les paramètres `stationID`, `Year`, `Month`, `Day`) pour obtenir un fichier `.CSV` qui contient les prévisions horaire pour une journée donnée. Le fichier `.CSV` retourné contient les prévisions pour la semaine dans laquelle la journée demandée se trouve et contient plus d'information que ce que nous avons besoin. Ainsi, le fichier `.CSV` retournée par la requête est traité et seulement les informations dont nous avons besoins sont conservées, puis retournées au client.

Méthode(s):

- `#convertCtoF(C)`

Méthode qui converti des degrés celcius en degrés fahrenheit. Utilisé pour le calcul de l'indice de chaleur.

Source du calcul: https://d138zd1ktt9iqe.cloudfront.net/media/seo_landing_files/c-to-f-formula-1638963368.png

- `#convertFtoC(F)`

Méthode qui converti des degrés fahrenheit en degrés celcius. Utilisé pour le calcul de l'indice de chaleur.

Source du calcul:

https://www.calculatorsoup.com/images/calculators/converters/fahrenheit_to_celsius_c18.png

- `#getFeltTemperature(trueTemperature, relativeHumidity, windSpeed)`

Méthode qui calcule la température ressentie à partir de la température réelle ambiante, l'humidité relative (pour l'indice de chaleur) et la vitesse du vent (pour le refroidissement éolien).

Sources: https://fr.wikipedia.org/wiki/Temp%C3%A9rature_ressentie,
<https://www.weather.gov/ama/heatindex>.

- `#getPastHourlyRelevantInfo(objRow)`

Méthode qui construit un tableau représentant une rangée des données recueillies contenant que les informations nécessaires, soit la date et l'heure, la température réelle (°C), la température ressentie (°C), les caractéristiques visuelles de la température (le temps qu'il fait), l'humidité relative (%), la direction du vent en dizaines de degrés, la vitesse du vent (km/h) ainsi que la pression atmosphérique (kPa).

- `fetchPastHourlyForecast(stationID, year, month, day)`

Méthode qui construit la requête

https://climate.weather.gc.ca/climate_data/bulk_data_e.html (avec les paramètres `stationID`, `Year`, `Month`, `Day`) afin d'obtenir les prévisions horaires d'une journée passée. Les données retournées par l'API `fetch` sont sous format `.CSV` et afin de pouvoir efficacement accéder

aux données il faut parser et traduire le CSV en objet JavaScript. Nous utilisons une classe, que nous avons nous-même programmé, nommée `ObjParser (Utils/Parser.js)` qui parse et interprète du contenu `.CSV` pour le transformer en objet JavaScript convivial à utiliser. Une fois l'objet JavaScript entre nos main, nous passons sur chaque rangée de l'objet, vérifions que la journée dans la rangée correspond avec celle spécifiée et construisons un nouvel objet ayant des rangées contenant les données qui nous sont pertinentes (`getPastHourlyRelevantInfo(objRow)`).

- `getPastHourlyForecastHeader()`

Cette méthode ne fait que retourner un tableau de chaîne de caractère représentant le texte de l'entête de chaque colonne à afficher.

StationMapping.js

Ce fichier contient une classe s'occupant de fournir une interface fonctionnelle pour aller chercher des informations sur les associations entre les identifiants de stations (une station peut avoir plus qu'un identifiant) et le lien RSS, contenus dans le fichier `station_mapping.json`.

Import(s):

- Module `fs` de Node.js pour lire des fichiers.

Méthode(s):

- `load()`
Charge le fichier `station_mapping.json` et construit l'objet JavaScript avec `JSON.parse()`.
- `getFromCode(code)`
Obtient l'entrée qui correspond avec le code de la station météo (exemple: `YUL` représente l'aéroport de Montréal qui a les identifiants `5415` et `51157`, et qui a un flux RSS à l'adresse `"https://meteo.gc.ca/rss/city/on-137_f.xml"`).
- `getFromID(id)`
Obtient l'entrée qui correspond avec le ou les identifiants de stations météo.

Prévision

StationForecast.js

Ce fichier JavaScript définit une classe « `StationForecast` » qui contient une méthode statique async « `getForecast` » pour obtenir les prévisions météorologiques d'une station donnée en utilisant son ID.

Import(s) :

- Module `fs` de Node.js pour lire des fichiers.
- `DOMParser` : Module `xmldom` pour analyser les chaînes `XML`.

Méthode(s) :

- `getForecast(stationID)`
On commence pas initialisé la valeur de retour « `resultArr` » et on lie le fichier « `station_mapping.json`

». On compare le stationID qu'on reçoit avec les données reçues du fichier JSON et on store le bon « URL » de la station choisie dans la valeur appropriée. On va utiliser la valeur « url » pour faire un « await fetch » sur le fichier xml en ligne. Une fois qu'on reçoit un String on utilise DOMParser pour convertir la réponse « String » en valeur que nous pouvons naviguer. En utilisant les fonctions du DOMParser (comme getElementByTagName et getAttributes) nous sommes capables d'aller chercher les valeurs du titre, lien, la mise à jour, la prévision actuelle, l'alarme et les prévisions des prochains jours (incluant les sommaires simples et détaillés). Pour les valeurs de titre, lien, la mise à jour, la prévision actuelle et l'alarme, nous naviguons de manière simple en utilisant les fonctions mentionnées plus haut. Pour les prévisions des prochains jours, on utilise une boucle « for » afin de naviguer chaque entrée en ignorant les entrées utilisées pour les informations déjà capturées. Chaque entrée est insérée dans une valeur tableau. Quand toutes les informations sont insérées dans « resultArr », on fait le retour de celle-ci. On utilise un « then/catch » si le site n'existe pas et on vérifie avec un « if » si l'url n'est pas vide.

Voici l'arborescence globale de l'application :

Structure de Répertoires

Client

- **Configuration et dépendances**

- .gitignore
- .prettierrignore
- .prettierrc
- babel.config.cjs
- jsconfig.json
- package-lock.json
- package.json
- vue.config.js

- **Archives**

- lab01/

- **Public**

- favicon.ico
- index.html
- js/
 - bootstrap.bundle.min.js
- Laboratoire_1_-_Enonces-20240516/
 - Lab1_CSV.zip
 - Lab1_CSV/

- **Src**

- App.vue
- main.js
- Assets/

- **Components/**
- **Css/**
- **Html/**
- **Js/**
 - **Stations/**
 - **Utils/**

Server

- **Configuration et dépendances**

- `.gitignore`
- `eslint.config.mjs`
- `package-lock.json`
- `package.json`

- **Data**

- `station_mapping.json`

- **Src**

- `index.js`
- `routes.js`
- **Stations/**
- **Utility/**

2. API et parser

ParagraphUtil.js

Une création d'équipe qui permet de traiter les informations reçues pour, ensuite, créer un paragraphe.

DOMParser (xmlDom)

Une création de xmlDom qui a permis convertir la réponse String du site en valeur xml de façon que l'on puisse naviguer facilement.

ObjParser

La classe `ObjParser` (du fichier `Utils/Parser.js`) est une classe utilitaire programmée entièrement par nous-même pour la conversion de données `.CSV` en objet manipulable JavaScript. Le processus se fait en 2 étapes:

1. Parser les données `.CSV` en tableau de rangées avec `ObjParse.parse(data)`.
2. Construire l'objet JavaScript qui associe chaque valeur de chaque rangée à l'en-tête correspondante avec `ObjParse.csvToObj(data, startAt)` (le paramètre `startAt` permet à l'utilisateur d'indiquer à quelle ligne les données et l'en-tête commencent puisque certains fichiers `.CSV` ont des commentaires au début). Si une rangée n'a pas le même nombre de valeur qu'il y a d'en-tête, la ligne est ignorée et un message y est écrit dans la console.

2.1 Gestion des données manquantes

Informations météo pour une journée donnée passée

Au niveau de l'interface, si le tableau de rangées retournée par le serveur ne contient aucune rangée, alors l'interface demande à l'utilisateur de choisir une nouvelle date.

3. Gestion des conditions d'erreur

Informations météo pour une journée donnée passée

- Au niveau front-end et back-end , nous utilisons un `if` sur la réponse pour vérifier que l'échange c'est bien déroulé. Nous n'avons pas utilisé `.catch(...)`, ni de clauses `try-catch`. Au niveau backend

Prévision

- Front-end (ForecastWeek.js): On utilise un `if` pour vérifier si la réponse du Back-end est bonne.
- Back-end (StationForecast.js): On utilise un `then/catch` si le site web n'existe pas ou ne renvoie pas un fichier xml. Ensuite, on utilise un `if` pour vérifier si l'url n'est pas vide.

4. Code dorsale et module ES6

L'une des contraintes du laboratoire portait sur l'utilisation du systèmes de modules ES6 et non CommonJS. Cela requiert de mentionner le mot clé `import` et non `require` pour utiliser une librairie ou simplement un fichier externe.

Parmis les modules importés on retrouve par exemple les différents modules javascripts liés aux composants tels que *NavigationMenu* ou *DataStatMenu* qui sont importés dans le fichier *Vue.js*, le module du composant parent *App.vue*

On peut aussi citer les librairies publiques telles que [papaparse](#).

Pour exporter nos modules nous avons optés pour deux façons de faire. Tout d'abord celle utilisée par *Vue* pour définir le module javascript d'un composant avec `"export default {}"`. Nous avons aussi utilisé la syntaxe `"module.export"` pour exporter des variables ou des fonctions (classes).

Nous avons par ailleurs régulièrement rencontré un problème d'import lié à la syntaxe utilisée au niveau du fichier d'import. En effet en omettant le `.js` à la fin du fichier dans le nom du chemin (relatif ou absolu) le module exporté n'était pas correctement trouvé et l'import ne fonctionnait pas.

Conclusion

Le point principale du second laboratoire a été la mise en place de l'application dorsale et toutes les modifications qui en ont découlées. Nous avons du revoir l'architecture globale du projet et ajuster le système d'import pour convenir à la contrainte technique du laboratoire. La mise en place du server backend a permis d'organiser la communication avec les APIs externes pour ajouter des fonctionnalités à notre application. Toujours en adaptant la structure du cadriciel [Vue.JS](#) nous avons créer de nouveaux composant pour chaque nouvelle fonctionnalité.